Hybrid Feature Selection and Multi-Model Evaluation for Android Malware Classification

Mohammad Bakeer

Supervised by: Dr. Malik Al-ESSA

1. Introduction

This study is based on the **CICMalDroid2020** dataset, using the option of 470 features. The 470-feature set was selected due to the dynamic features it has, which could improve detection rates by analyzing malware behavior during execution. In addition to the computational feasibility compared to the 50,621-feature set.

The dataset owners' research paper [1] was utilized through this study, especially through the feature selection phase.

2. Methodology

a. Preprocessing

Train/Test split ratio (70% - 30%) was used according to paper [1], where the accuracy of this approach outperformed other ratio sets. duplicates were dropped before the splitting to ensure no duplicates across the train and test. Then I cleaned the data by checking null and infinite values, also dropped zero variance columns as they contain constant values, providing no information for the ML models.

b. Feature Selection: Filter Methodes

In features selection I used the approach proposed by the paper [1], where I first used Variance Threshold 0.1 to remove all low-variance features, followed by Mutual Information to select the top 40 features. However, my selected set shared only 17 features with the paper's results. This difference stems from the authors applying variance filtering to static and dynamic features separately before concatenation, rather than on the combined set. Both my MI 40-features set and the paper's top 40 features were used individually for model evaluation, delivering a comparison between the different approaches.

c. Feature Selection: Wrapper Methods

First I applied **Recursive Feature Elimination** on the paper's top 40 features to reduce the number to 25, then I used **Backward Elimination** to choose the top 20 features. Both wrapper methods were evaluated on XGBoost classifier as it will be used in ML training and also takes less time than both MLP and SVM classifiers especially during Backward Elimination. RFE runtime was less than a minute, but the problem was with BE as it took 235 seconds on eliminating 5 features only which is the reason of choosing the number 20 due to computational cost.

d. Machine Learning Models

Three machine learning models were used: **SVM**, **MLP** and **XGBoost**. Each model was evaluated on three feature sets: the 20-feature wrapper subset, the 40-feature Mutual Information (MI) set, and the paper's original top 40 features. The paper's top 40 features consistently outperformed the other sets for SVM and MLP, achieving 0.7169 and 0.8814 accuracy respectively. XGBoost performed best on the MI 40-feature set (0.9346) but was nearly identical to the 20-feature set (0.9283), making the 20-feature set preferable due to lower complexity.

Model	Feature Set	Accuracy	Precision	Recall	Macro F1	Train Time (s)	Predict Time (s)
SVM	(20)	0.5428	0.5855	0.4361	0.4234	16.546	8.740
SVM	MI (40)	0.4615	0.6564	0.3419	0.3240	18.282	9.067
SVM	**Paper top 40**	**0.7169**	**0.7047**	**0.6404**	**0.6524**	**11.030**	**8.846**
MLP	(20)	0.7212	0.6944	0.6549	0.6649	2.876	0.015
MLP	MI (40)	0.7915	0.7513	0.7393	0.7442	20.533	0.017
MLP	**Paper top 40**	**0.8814**	**0.8650**	**0.8550**	**0.8594**	**312.163**	**0.077**
XGBoost	(20)	0.9283	0.9139	0.9121	0.9129	3.023	0.052
XGBoost	**MI (40)**	**0.9346**	**0.9204**	**0.9180**	**0.9190**	**6.577**	**0.052**
XGBoost	Paper top 40	0.9265	0.9132	0.9102	0.9116	7.481	0.041

e. Hyperparameter Optimization using GridSearchCV

GridSearchCV with 5-fold cross-validation was applied to all three classifiers. **SVM** and **MLP** were tuned on the paper's top 40 feature set as it yielded their best baseline results, while XGBoost was tuned on the 20-feature wrapper set since it achieved nearly identical performance to the MI 40-feature set (0.9283 vs 0.9346) with lower complexity. All hyperparameter configurations evaluated during the search process were selected based on values documented in existing research.

The following searching grid parameters were explored for **SVM**:

- C: [0.1, 1, 10]
- kernel: ['rbf', 'linear']
- gamma: ['scale', 'auto']

These parameters were adopted from Aldhafferi [2]. For **MLP**, 4 parameter lists were chosen based on paper [3]:

- hidden_layer_sizes: [(50,), (100,), (50, 50)]
- max_iter: [500, 800, 1000]
- activation: ['relu', 'tanh']
- alpha: [0.0001, 0.001]

For XGBoost, the following parameters grid were selected:

- n_estimators: [50, 100, 200]
- max_depth: [3, 6, 9]
- learning_rate: [0.01, 0.1, 0.2]

The table below summarizes all GridSearch results:

MODEL	BEST HYPERPARAMETERS	CV MACRO-F1	ACCURACY	PRECISION	RECALL	MACRO F1	GRIDSEARCH TIME (S)	PREDICT TIME (S)
SVM	C=10, gamma='scale', kernel='rbf'	0.7604	**0.7918**	0.7794	0.7408	0.7555	308.988	5.110
MLP	activation='relu', alpha=0.0001, hidden_layer_sizes=(50,50), max_iter=800	0.8605	**0.8962**	0.8808	0.8688	0.8743	5331.364	0.092
XGBOOST	learning_rate=0.2, max_depth=6, n_estimators=200	0.9106	**0.9271**	0.9113	0.9111	0.9111	146.359	0.042

3. Results & Analysis

The paper's top 40 features gave the best results for SVM and MLP, while XGBoost performed best on the MI 40-feature set but nearly identically on the 20-feature wrapper set. While feature selection helped reduce the complexity and computational cost, the paper top 40 features set was highly effective with no need to reduce them with wrapper methods in most models, except for the XGBoost it really helped reduce the complexity.

Filter methods were the most important process as it was the way the paper [1] reduced the features to 40 since it allows us to eliminate high number of irrelevant features based on clear standards like low variance features. Wrapper methods are great if we are sure which model we will adopt as it is highly expensive in time and resources, so it would be hard to test different models.

GridSearchCV was really effective as it helped remove the need to manually test each parameter to find the best results. SVM had a significant improvement from the hyperparameter tuning as we were able to raise the accuracy by +7.49% gain.

In general, XGBoost proved to be the most effective architecture, demonstrating high resilience to feature reduction and achieving above 92% accuracy across all feature sets. As a result, XGBoost with the 20-features set is highly recommended as it provides almost the highest accuracy with half of the features at the same time, significantly reducing the computational cost .

4. References

[1] S. Mahdavifar, D. Alhadidi, and A. A. Ghorbani, (2022) "Effective and efficient hybrid Android malware classification using pseudo-label stacked auto-encoder".

[2] N. Aldhafferi, "Android malware detection using support vector regression for dynamic feature analysis," *Information*, vol. 15, no. 10, p. 658, Oct. 2024. [Online]. Available: https://doi.org/10.3390/info15100658

[3] O. Jurečková, M. Jureček, and M. Stamp, "Online clustering of known and emerging malware families," *arXiv preprint arXiv:2405.03298*, May 2024. [Online]. Available: https://arxiv.org/abs/2405.03298